Research 240

Digital Data Broadcasting and Management

Edward Y. Chang
Electrical & Computer Engineering
University of California
Santa Barbara, CA 93106
echang@ece.ucsb.edu

Chen Li
Department of Computer Science
Stanford University
Stanford, CA 94305
chenli@db.stanford.edu

Abstract

The FCC has mandated TV broadcasters to transmit data in the digital format by the year 2005. Digital broadcasting enjoys high transmission fidelity, which increases the number of broadcast channels that can occupy the same spectrum. The resulting total broadcast bandwidth has been estimated at 10 Gbps, which is 1,000 times the bandwidth that a typical Internet end user can get today. The flexibility and extensibility of an all-digital system raise many interesting possibilities. In this study, we show that combining digital broadcasting and the Internet effectively can enable applications such as Interactive DTV and can support a wide range of data services. We study issues including data characterization and data management at the server and the client side, investigating some data service as well as data subscription models.

1 Introduction

The Federal Communications Commission (FCC) has mandated TV broadcasters to transmit TV in the digital format by the year 2005. The flexibility and extensibility of an all-digital system raise a wide range of interesting database-research possibilities. Two applications that will benefit from research in the database community are data broadcast and Interactive TV, and each of these applications raises a question:

- *Data Broadcast*: Given the broadcast nature of the medium, can data broadcast be used for other types of information dissemination?
- *Interactive DTV*: What types of interaction are possible within the context of television, and what infrastructures are needed to support interactivity efficiently?

The market of data broadcast and interactive DTV has huge potentials. A recent study by a Dataquest report [1] shows that the number of people watching TV while simultaneously surfing the Web has risen from 8 million in 1998 to 27 million in 1999. Many of these "telewebbers" use the Internet to look up more information about a news story or to chat with other viewers

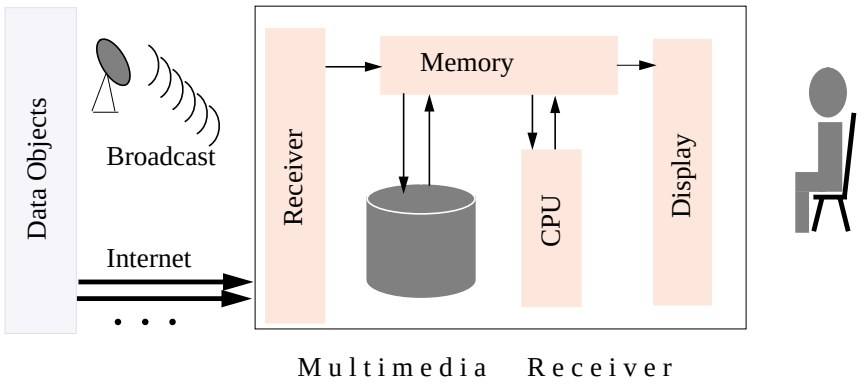


Figure 1: The Multimedia Data Delivery Pipeline.

and vote in online polls while they are watching a show. The increasing number of telewebbers attests to the potential for including TV-like functionality in the PC and Internet functionality through television and interactivity in TV programs. In addition, according to Forrester Research (http://www.forrester.com/), interactive TV will be worth $20 billion in advertising, commerce and subscription revenue by the year 2004. Recognizing the potential revenue, companies such as AOL, Intel and Microsoft have recently started developing technologies and standards for supporting data broadcast and interactive TV.

Digital broadcast enjoys high transmission fidelity, which increases the number of broadcast channels that can occupy the same spectrum. With the capability of supporting more than 500 channels and with each channel having 19 Mbps bandwidth, the total broadcast bandwidth can amount to 10 Gbps, which is 1,000 times the bandwidth that a typical Internet end user can get today. We see the Internet and broadcasting as two complementary data-delivery mechanisms. The Internet may help a broadcast program become interactive, and a broadcast channel may help alleviate the traffic on the Internet and thereby make the Internet more efficient. The situation is analogous to that of an efficiently run public transportation system (data broadcasting), which can bring people to desired locations with a short delay and thereby reduce traffic. The reduced traffic, in turn, makes the private vehicles on the road (the Internet) more efficient means of transportation.

But how do we make data broadcasting an effective means of transportation and what technical issues can the database community help resolve? To present the problems, Figure 1 depicts a data delivery pipeline. On the left-hand side of the figure, data objects are transmitted via broadcast and/or Internet channels to a receiver. The receiver receives network packets and saves them in its main memory. If the data are not needed shortly (e.g., the user does not need the data live), the data are written to the user's local disk to conserve memory. The data are made available to the CPU before they are needed. The CPU processes (e.g., parses, filters, computes, decodes, renders, etc.) the data in main memory. Finally, the processed data are presented to the viewer on the right-hand side of the figure.

There are many unanswered research questions at both the server and the receiver side. At the server side, some unresolved issues are

- Should data be placed on the Internet or should it be broadcast? What are the criteria used to decide whether to do one or the other?
- If a data object is to be broadcast, how should its broadcast be scheduled to maximize profit?
- If multiple objects are to be broadcast, how should these objects be packetized so that the delay of delivery is minimum and the channel utilization is maximum?
- What is a proper data broadcast format? Where should the meta data be stored? How can a search take place in broadcast channels?
- How are transactions affected by data broadcast? What should concurrency be controlled when broadcast data are updated by different users?
- How should broadcast data be encrypted so that only authorized users (e.g., users who pay) can access them?

At the client (receiver) side, some unresolved issues are:

- What is the data subscription model? How can a user convey his/her interest to a broadcaster? Should a user know the source of the data?
- How can a receiver manage its local resources effectively to support various data-processing tasks and different interactive scenarios?
- If data arrive from multiple channels, both broadcast and Internet, how can related data be synchronized properly? How can data that are related, shared or unrelated be indexed and stored?

To further complicate matters, the design of the servers and clients faces at least three challenges:

- Not all data are equal. Data objects can be continuous, textual, binary, and so forth. Different data have different characteristics.
- Not all data servers/receivers are equal. Different servers/receivers may have different available hardware resources and network bandwidth. A server must decide how its data are disseminated to minimize its data delivery cost. For instance, if a server cannot afford buying hundreds of workstations and installing a high-bandwidth Internet infrastructure to service thousands of interactive users, it may consider broadcasting some data in the air. On the receiver side, performance should degrade, if inevitable, in a graceful manner under the receiver's resource constraints.
- Not all subscribers are equal. Different viewers may have different viewing preferences. To measure a viewer's satisfaction, we must allow the user to specify what is important to the user and what is not. We thus assign to each data object a QoS ("Quality of Service") parameter. A viewer can assign a QoS value to each data object either implicitly or explicitly.

The magnitude of the problem is daunting. In this paper, we focus our discussion on three areas:

1. *Data Characterization*: We show how to characterize different data objects using their sizes, update frequencies, latency sensitivities, and other parameters. We also discuss how to quantify the value, cost and quality of service (QoS) requirement of a data object.

2. *Data Service*: We study the problem of determining what data are more suitable for broadcast, and how frequently an atomic data object should be broadcast so that its latency in arriving at the subscribers is satisfactory and the information it provides is up-to-date.

3. *Data receivers*: We discuss how a client conveys his/her needs to the server and how the server informs the client to receive data. We study how a client manages its resources (memory, disk) to support fast-scans without hiccups during the broadcast of programs. We propose disk scheduling and data placement policies. We also propose a QoS-adaptive scheme that considers user preferences for maximizing the total quality of service.

Shipping merchandise and people from one location to another has been a challenging problem since the start of human history. The data transportation problem is more complicated because of the additional issues discussed above. We believe that data broadcasting and management is an area that will inspire much database research and generate much commercial interest in the near future.

2 Related Work

Work has been done in two areas related to data broadcasting: broadcast disk and Internet broadcast [2, 9]. Both broadcast disk and Internet broadcast attempt to *push* frequently accessed data to a large number of clients while allowing clients to *pull* data. However, most analytical models have been static because they made the following assumptions:

1: All data are equal,

2: All receivers are the same,

3: All users are assumed to be invariant in their QoS requirements, and

4: All data are read-only.

In this paper, we show that relaxing these assumptions opens up many interesting data management problems, and we present some preliminary suggestions regarding how to tackle some of these challenges.

In addition, DTV broadcasting introduces unexplored problems in four new areas:

1: Broadcast scheduling: A digital channel can be subdivided into subchannels. Multiple data objects can be transmitted at the same time rather than in an interleaving manner. Intelligent data packetization schemes can be devised to utilize the channels more efficiently and to minimize the latency that clients experience.

2: Cost modeling: Contrary to the Internet, air broadcast is not free. Channel allocation and capacity management must take cost into consideration.

3: Global load balancing: Air and Internet channels have different peak hours. An application should explore off-hour bandwidth to improve its overall efficiency at reduced cost.

4: Data synchronization: Related data may arrive at a client from more than one channel, broadcast or Internet, at different times. Spatial as well as temporal indexes can help organize and synchronize related data.

Finally, many studies have proposed Internet-based media server architectures (e.g., [4, 5, 6, 10, 13, 15, 16, 17, 20]), in which a server schedules its resources (e.g., memory and disk bandwidth) to service interactive VCR-like requests, such as pause, replay and fast-forward. However, because of the typical long end-to-end Internet transmission delay and the Internet's limited bandwidth, it is practically infeasible for the server to support "real-time" VCR-like interactive operations for tens of thousands of simultaneous users. We belive that with data broadcasting and with a smart receiver [3], interactive TV can be supported effectively.

3 Data Characterization

Many different data objects can be delivered from servers to clients, among them video/audio clips, HTML texts, and images. Data objects can be continuous, textual, binary, and so forth. In this section we discuss how to characterize data objects using their *size*, *update frequency*, *delay sensitivity*, etc. Data characteristics are considered in determining data scheduling and packetization policies at the server side (Section 4), and are considered in managing resources at the client side (Section 5).

3.1 Size

The size of a data object is measured by the amount of storage space (and thus the transmission bandwidth) the object needs. The size of a data object can vary from a few bytes to several gigabytes. Continuous data (video streams) can be voluminous. For example, a one-hour 19 Mbps HDTV program takes up 8.55-GByte space. Textual data (captions, HTML and XML documents, etc.) in general occupy far less space than continuous data. Other data, such as images and applets, can be very copious or scant.

3.2 Update Frequency

We use F_i to represent how frequently the ith data object, D_i, is updated. Static data objects, such as movies and CDs, do not change over time. Once a static object is delivered, the server does not need to worry about the data being out of date. Dynamic data objects, such as stock quotes, sports game scores and the seat availability on a flight, can change over time but can also become static. For instance, once the outcome of the last U.S. Open Tennis Tournament has been decided, it will not change.

Different dynamic data objects have different update frequencies. For instance, a stock quote during daytime may change every few seconds, while the score of a sports game does not change as often. To describe the update behavior of data object D_i, we assign a stochastic process to the object. Stock quotes, for instance, can be modeled as a uniform process. The score of a sports game may observe a Poisson process during the game, and becomes constant once the game is over.

To characterize the *freshness* of an object D_i, we use the *age* τ_i of the object. The age of

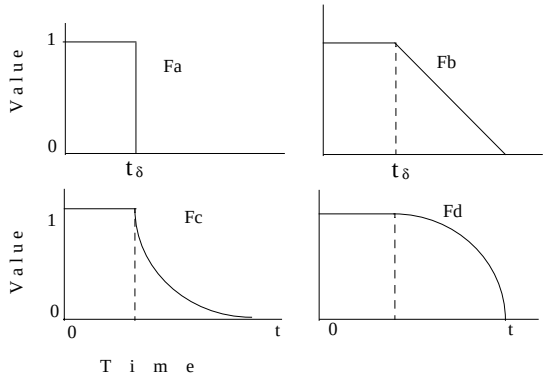


Figure 2: Delay Function Examples.

a static object, such as a movie and a CD, remains zero since it is always fresh. The age of a dynamic object D_i at time t is defined as follows:

$$\tau_i = t - T_{U_i}, \tag{1}$$

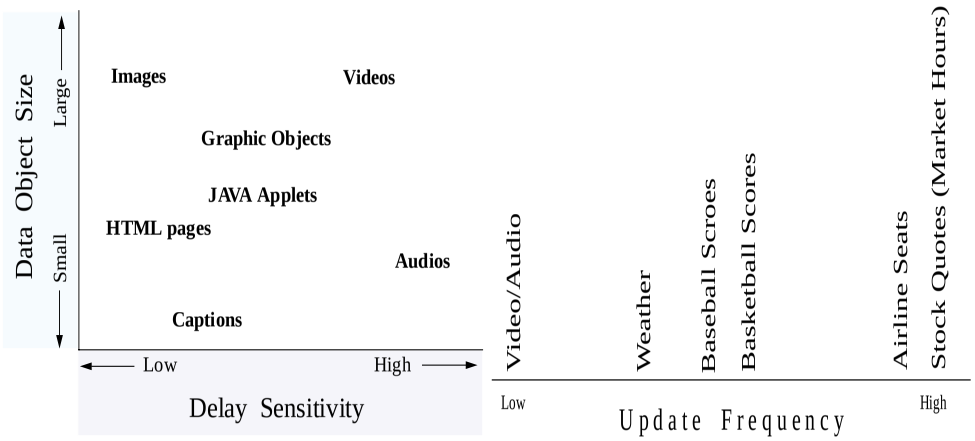
where T_{U_i} denotes the time when the object was last updated.

Age is a critical factor to determine the value of an object. An ageless object is always fresh, so its value is constant. The value of a dynamic object often decays with age. For instance, a stock quote with a one-minute delay may not be useful for making real-time trading decisions. Out-of-date seat availability information on a flight may result in double booking. The server should attempt to make the broadcast data as fresh as possible to maximize the usefulness of the data.

To quantify the usefulness of a data object, one can specify a value function for it. Figure 2 shows four representative examples. The x-axis in the figures depicts the update delay (age); the y-axis depicts the value of the object normalized to from zero (worthless) to one. Function F_a depicts an object that can tolerate update delay up to a period of time, t_δ, and then becomes worthless. Function F_b shows the value of an object that decays linearly after t_δ. Functions F_c and F_d have other value-decay patterns. Function F_a is a good value function for a stock quote. Function F_b may be a good value function for the score of a baseball game since a slight delay is tolerable. Function F_c may be a good value function for traffic reports. Function F_d, which maintains a high value for a long period of time before rapidly decaying, may be a good value function for weather reports. Without losing generality, we define the value for object D_i as a function of its age: $f_i(\tau)$, where τ denotes the age of the object.

3.3 Delay sensitivity

There are two types of delay, *initial delay* and *hiccup*. Initial delay is the time between when an object is requested by a client and when the object is delivered to the client. Hiccup is used to describe the delivery interruption of continuous data objects. For instance, a client waits for five minutes to start receiving a movie that she requested, thus the initial delay for watching



(a) Size and Delay (b) Update Frequency

Figure 3: The Characteristics of Data.

the movie is five minutes. If the data cannot be delivered to satisfy the real-time playback requirement of the movie after it is started, every disruption to the playback is counted as one hiccup. Textual data, images and applets do not have real-time constraints if they are processed independently of a continuous media stream. Any data, however, must satisfy real-time requirements if the data must be synchronized with a continuous stream. For instance, the display of the captions of a movie must be in synchronization with the audio track.

Value functions can also be used to characterize the effect of initial delay and hiccup. For instance, we can use value functions similar to those used for data freshness, which describe the value of a data object as time changes, to describe how the value of an object changes as its delay/hiccup increases.

Remark:

In addition to size, update frequency and delay sensitivity, data broadcasting also needs to consider if a data object has real-time delivery constraints and if an object must be reliably delivered. A/V data when they are played back live have real-time constraints, but dropping a few A/V bits may be acceptable. On the contrary, delivery software typically does not need to meet real-time requirements, but the entire software is useless if it drops a bit.

Figure 3(a) shows the sizes and delay sensitivity of some representative objects. The x-axis in the figure represents the delay sensitivity from high (on the left-hand side) to low (on the right-hand side); the y-axis gives the size of an object. At one extreme, videos are large and have strict real-time constraints. At the other extreme, captions are small and not very sensitive to delay. Figure 3(b) shows the update frequencies of some example data objects.

Finally, Table 1 summarizes the data parameters that we have discussed to characterize

data object D_i. We also list parameters that will be introduced in the later sections.

Parameters	Meaning
$f_i(\tau_i)$	Value function for age
$g_i(\tau_i)$	Value function for delay
S_i	Data size
F_i	Data update frequency
τ_i	Age
T_{U_i}	Last update time
CB_i	Broadcast cost (per byte)
CI_i	Internet transmission cost (per byte)
R_i	Revenue

Table 1: Parameters for data object D_i.

4 Data Services

Suppose a server has M data objects to deliver to its clients. It must make the following decisions:

1: What broadcast frequency to use: If a data object should be broadcast, how often should it be broadcast to maximize its profit (defined shortly)?

2: What kind of packetization to use: How should the objects be organized in order for them to be transmitted simultaneously?

In this section we discuss how the server makes these decisions.

4.1 Broadcast Scheduling

Given data objects and a time period, a server broadcasts these objects periodically to maximize its profit. How many times each object should be broadcast depends on its size, cost, delay sensitivity, and other characteristics. In this section we formulate the problem of determining broadcast frequencies of different data objects.

Assume that a server needs to broadcast M data objects $D_1, \ldots, D_M$ in a time period T. Let the broadcast bandwidth be BB, and the cost of broadcasting a byte be C_{byte}. Let N_i denote the number of times the data object D_i is broadcast during the time T. The scheduling objective function of the server can be modeled as

$$Max \sum_{1}^{M} (R_i - CB_i), \tag{2}$$

where R_i denotes the revenue of broadcasting D_i, and CB_i denotes the cost of broadcasting D_i. We would like to schedule the broadcast in a way that the total profit (revenue minus cost) is maximized.

One way to model the revenue of broadcasting D_i is as follows:

$$R_i = K_i \times f_i(\tau_i) \times g_i(\tau_i) \times F_i. \tag{3}$$

That is, the revenue of broadcasting D_i is characterized by the amount of money each request for the object is worth multiplied by the number of requests F_i. The worth of a request can be measured by the quality of the service, which is represented by $f_i(\tau_i) \times g_i(\tau_i)$, where τ_i represents the average delay caused by the broadcast schedule, i.e., $\tau_i = T/N_i$. We use $f_i(\tau_i)$ to gauge the freshness of the data and $g_i(\tau_i)$ to gauge the effect of the delay on the service. Finally, we multiply the quality with a constant K_i to translate quality into payment.

The cost of broadcasting D_i depends on the usage of the bandwidth, which is the number of bytes transmitted times the transmission cost per byte. We can thus express CB_i as

$$CB_i = C_{byte} \times N_i \times S_i. \tag{4}$$

Finally, the total amount of data broadcast cannot exceed the channel capacity BB. This constraint can be written as

$$\sum_{i}^{M} (N_i \times S_i) \leq BB \times T. \tag{5}$$

Replacing R_i and CB_i in Equation 2 with the right-hand side of Equations 3 and 4, the objective function can be rewritten to the following form:

$$Max\{\sum_{1}^{n} (K_i \times f_i(\tau_i) \times g_i(\tau_i) \times F_i - C_{byte} \times N_i \times S_i)\} \tag{6}$$

4.1.1 Parameter Relationship

To show some simple relationships between parameters, we temporarily assume that the network bandwidth $BB = \infty$. (We will install this constraint shortly.) Solving these N_i's requires precise knowledge of each $f_i(\tau_i)$ and $g_i(\tau_i)$, which we may not have. But we know that the value functions $f_i(\tau_i)$ and $g_i(\tau_i)$ are monotonically decreasing. That is, the less frequently a data object is broadcast, the longer the delay to keep the data value up-to-date at the client side, and thereby the less satisfied a user is. Since $f_i(\tau_i)$ and $g_i(\tau_i)$ are both monotonically decreasing with respect of τ_i, R_i has the same property. A representative R_i can thus be written as:

$$R_i = M_i - \frac{F_i \times K_i \times T}{N_i^{\beta_i}}, \tag{7}$$

where M_i is the maximum revenue that the server can attain by broadcasting data D_i infinite number of times and K_i is a constant factor. Since the object can only be broadcast a limited number of times, its revenue is M_i minus a factor $\frac{F_i \times K_i \times T}{N_i^{\beta_i}}$, where β_i is a positive number. The larger β_i is, the faster the revenue decays with update and transmission delay.

To achieve the maximum profit, the first partial derivative of each N_i should be equal to 0. That is,

$$\frac{F_i \times K_i \times T \times \beta_i}{N_i^{\beta_i+1}} - C_{byte} \times S_i = 0. \tag{8}$$

Isolating N_i on the left-hand side of the equation, we obtain

$$N_i = \Big(\frac{F_i \times K_i \times T \times \beta_i}{C_{byte} \times S_i}\Big)^{\frac{1}{\beta_i+1}} \tag{9}$$

Based on Equation 9, we can observe the following relationships:

1. $F_i \times T \uparrow$: If the number of viewers for data object D_i increases, N_i should also increase. Intuitively viewed, if more clients request object D_i, it should be broadcast more often to meet the clients' requests.

2. $C_{byte} \times S_i \uparrow$: As the size of the data D_i or the cost to broadcast increases, N_i should decrease.

3. $\beta_i \uparrow$: The smaller the value of β_i, the more significantly the changes in revenue and cost affect N_i. In other words, if a data object is updated frequently or is delay sensitive, changes in revenue or cost have a more significant effect on the optimal broadcast frequency. On the other hand, if a data object is not delay sensitive, changes in revenue or cost do not affect N_i much.

4.1.2 Limited bandwidth

Now, if we take the bandwidth constraint into consideration, the problem becomes more challenging. Assume we have a limited bandwidth $BB < \infty$, then different data objects will compete for the bandwidth. To determine which data objects should have more frequent broadcast schedules, we consider the marginal profit of broadcasting each object D_i. We then pick the D_i that can achieve the maximum marginal profit by following a greedy heuristic.

	1	2	3	...
D_1	$\$600K$	$\$200K$	$\$100K$	...
D_2	$\$200K$	$\$120K$	$\$80K$	...
D_3	$\$100K$	$\$30K$	$\$10K$	...

Table 2: Marginal broadcasting profits of three data objects.

Table 2 shows an example with three data objects, D_1, D_2 and D_3 to be considered. Suppose their sizes are $100MB$, $50MB$, and $10MB$, respectively. Each row depicts the marginal profit that can be achieved by increasing the broadcast frequency by one. For instance, when D_1 is broadcast once, the profit is $\$600K$. When we increase the frequency from one to two, the

marginal profit is $\$200K$. The marginal profit of increasing the broadcast frequency from two to three for D_1 is $\$100K$.

Suppose the initial available bandwidth $BB = 1GB$. First we compute the marginal profit ratio (MPR) of each object D_i, defined as its marginal profit divided by its size. The ratios for the three objects are: $MPR_1 = \$600K/100MB = \$6/KB$, $MPR_2 = \$200K/50MB = \$4/KB$, and $MPR_3 = \$100K/10MB = \$10/KB$. Therefore, we choose the object D_3 with the highest marginal profit ratio and broadcast it once.

After choosing D_3, the available bandwidth $BB = 1GB - 10M = 990MB$, so we can still choose programs to broadcast. We recalculate the marginal profit ratio of D_3, $MPR_3 = \$30K/10MB = \$3/KB$. We choose the object D_1 that has the highest marginal profit ratio $\$6/KB$ and broadcast it once. After that, the available bandwidth $BB = 990MB - 100M = 890MB$, and we continue choosing a data object to broadcast. We choose D_2 since it has the highest marginal profit ratio, $\$4/KB$. We repeat this process greedily to choose data objects to broadcast until we use up all the available bandwidth.

4.2 Packetization

Suppose M objects are to be broadcast and the ith object has size S_i and broadcast frequency N_i that has been determined based on the steps in Section 4.1. We need to determine how to package these M objects so that the broadcast bandwidth requirement is the minimum. We describe and analyze two schemes in this section: the Carousel scheme and the Pyramid scheme. One can develop a variety of schemes based on these two basic schemes.

4.2.1 The Carousel Scheme

The Carousel scheme transmits data objects (or DTV programs) in a fixed order in a periodic fashion. How to schedule a given set of periodic tasks to meet their deadlines has been widely studied in the real-time community. An effective scheme to test the schedulability of the tasks is Rate Monotonic Scheduling (RMS) [14, 18]. To formulate the broadcast schedule, we first select the broadcast period for each data object and compute the time it takes to transmit the object. Suppose we have M different data objects denoted as D_1 to D_M. We can compute the period to broadcast D_i, denoted as T_i, and the time it takes to transmit D_i, denoted as E_i, by using the following equations:

$$T_i = T/N_i, \qquad E_i = S_i/BB.$$

According to RMS, a set of M objects is packetizable (schedulable) if the following conditions can be met

- All objects are broadcast periodically,
- Objects are not synchronized with one another, and
- An object with a shorter period has a higher priority.

Then the packetizability of the data objects can be decided by the following test:

- Packetizable if $\sum_{i=1}^{M} \frac{E_i}{T_i} \leq n \times (2^{1/n} - 1)$
- Not packetizable if $\sum_{i=1}^{M} \frac{E_i}{T_i} > 1$
- Inconclusive if $1 \geq \sum_{i=1}^{M} \frac{E_i}{T_i} > M \times (2^{1/M} - 1)$

If RMS is inconclusive, we must employ an exact test procedure that can be summarized as follows:

$$a_{i+1} = E_M + \sum_{j=1}^{M-1} \lceil \frac{a_i}{T_j} \rceil E_j, \ where \quad a_0 = \sum_{j=1}^{M} E_j.$$

The test terminates when $a_i > T_M$ (not packetizable) or when $a_i = a_{i+1} \leq T_M$ (packetizable).

To improve the packetizability of a set of objects that cannot be packetized by the first attempt, one can divide a data object into sub-objects to improve the packetizability. This technique is referred to as "period transformation." Ideally, if one divides a K bits object into K subobjects (each subobject has only one bit) one can enjoy the optimal packetizability and achieve the optimal channel utilization. However, the header overhead of a subprogram that transmits a subobject prevents us from dividing an object too finely. (For instance, if the header takes up four bytes, the overhead of transmitting an one-bit object is horrendous.) Thus, care must be taken when dividing a data object into subobjects for transmission.

We use an example to illustrate how the Carousel scheme works. Suppose we have four data objects to be broadcast in four programs and their broadcast periods T_i and their transmission times E_i are shown in Table 3.

Data i	E_i (s)	T_i (s)	E_i/T_i
D_1	5	50	1/10
D_2	10	70	1/7
D_3	40	120	1/3
D_4	80	200	2/5

Table 3: Carousel Example.

The RMS test shows that we can packetize the first three programs since $\sum_{i=1}^{3} E_i/T_i = 0.576 < 3(2^{1/3} - 1)$. The RMS test, however, cannot determine if we can packetize all four programs since $1 > \sum_{i=1}^{4} E_i/T_i = 0.976 > 4(2^{1/4} - 1)$. We follow the exact test to check if all four programs can be transmitted to meet their deadlines. Following the steps of the exact test, the answer we get is no. Finally, we perform period transformation to convert D_4 into ten subprograms as shown in Table 4. As a result, we can packetize all programs to meet their transmission deadlines.

So far, we assume that a data object is broadcast periodically. If slack bandwidth exists, one can use the slack to schedule aperiodic tasks or to schedule addition broadcast for the data objects that have been updated before their transmission schedule.

$Data\ i$	E_i (s)	T_i (s)	E_i/T_i
D_4	8	20	2/5
D_1	5	50	1/10
D_2	10	70	1/7
D_3	40	120	1/3

Table 4: After Period Transformation.

4.2.2 The Pyramid Scheme

The Carousel scheme can suffer from long initial latency. For example, if a program is run once an hour, the worst-case initial latency for watching the program is one hour. To remedy this long-delay problem, we can employ the Pyramid scheme for multicast proposed by [12] to packetize data.

The idea of the Pyramid scheme is to divide a program into subprograms, whose sizes are in a geometrical series. If the first subprogram is K bytes, the ith subprogram is $2^i \times K$ bytes, and the total size amounts to S_i. The Pyramid scheme then broadcasts each subprogram repeatedly. Figure 4 shows an example where a program is divided into four subprograms. The x-axis of the figure is time. The first subprogram lasts for one unit of time, the second for two, the third lasts for four, and the last for eight. Each subprogram is repeatedly transmitted. The receiver receives all subprograms at the same time. The worst-case initial delay is the duration of the first subprogram.

Let us see how the scheme works via an example execution. Suppose a user starts downloading the data object at the time indicated by the A arrow in the figure. The receiver at the client side starts receiving all four subprograms at A. After the first subprogram is completely received, the receiver start the playback (indicated in the figure by the B arrow). When the first subprogram is consumed, the second subprogram will have completely arrived at the receiver and the playback resumes without a hiccup. This process continues until the entire program ends.

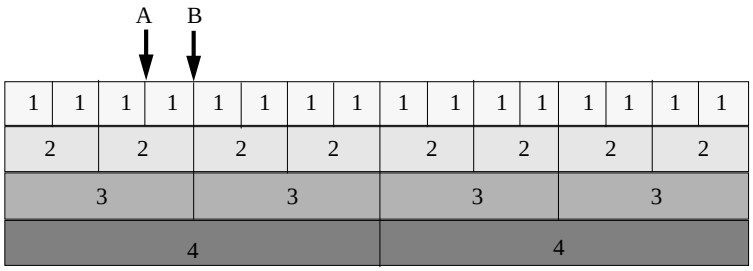


Figure 4: A Pyramid Examples.

The Pyramid scheme can enjoy very short initial latency if one is willing to trade bandwidth for performance. For a one-hour movie, we can divide the program into six subprograms that last for roughly $1, 2, 4, 8, 16$ and 32 minutes and transmit them repeatedly in six subchannels. In this case, we have a worst-case delay of only one minute. If we use the Carousel scheme to achieve the same one-minute latency, we have to broadcast the movie sixty times in an hour,

one minute apart, which takes up sixty subchannels. The Pyramid scheme is clearly more desirable. In general, the Pyramid scheme can be a big win when a data object has a large N_i and a large S_i.

Note that for the Pyramid scheme to work, the receiver must be able to receive multiple subprograms simultaneously. We thus need an intelligent receiver, which can buffer data properly and efficiently at minimum cost. Below, we discuss the design of such a receiver.

5 Data Receivers

A user must express his/her interests to the server so that the server knows what objects to broadcast and how frequently to broadcast these objects. The receiver's resource manager must be carefully designed to support interactive TV operations (i.e., pause, fast-forward and instance replay) and to synchronize conventional data with the playback of AV data. Since not all receivers have the same hardware configuration and not all users share the same viewing preferences, the receiver must be adaptive to both available resources and individuals' QoS requirements. In this section, we discuss

1: Data subscription: How a user conveys his/her needs to the server and how the server informs the user's system (i.e., the receiver) when and how to receive data.

2: Resource Management: When data arrive at multiple channels, the receiver must manage its resources effectively so that given a fixed amount of resources (e.g., memory and disk bandwidth), the utilization of the resources is the maximum. We propose disk scheduling and data placement policies.

3: QoS-adaptive model: When a receiver cannot handle all data processing tasks at hand, the receiver must degrade in a graceful manner. We propose a QoS-adaptive scheme that considers user preferences for maximizing the total quality of service.

5.1 Data Subscription

A user subscribes to data objects directly or indirectly. The user conveys a direct request by surfing channels or selecting programs in the broadcast program table and conveys an indirect request by invoking an application (e.g., interactive DTV) or an Internet interface. For example, one can start an interactive teleshopping application that needs audio, video, graphics and HTML texts. The invocation of the program in turn triggers the requests for various data objects.

Figure 5 depicts the data structures proposed by ATSC (http://www.atsc.org) to associate applications with data sources. They are:

- Program ID (PID): A PID is an identifier that uniquely identifies a DTV program in an MPEG2 broadcast stream (at the bottom of the figure).
- Network Resource Table (NRT): The NRT provides mapping between the logical channels provided by a data service and the physical channels.

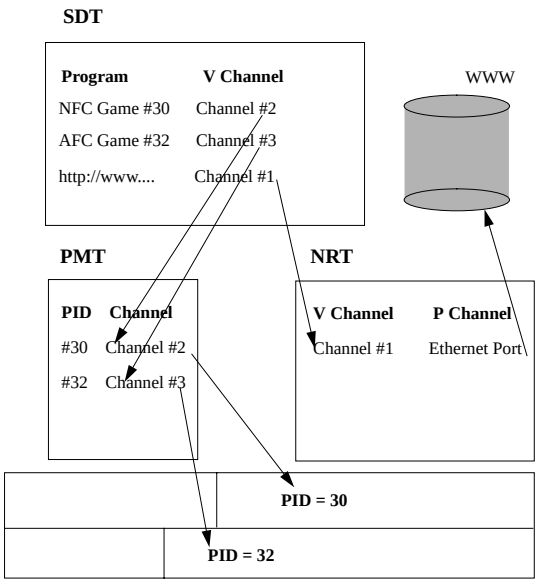


Figure 5: A Subscription Example.

- Program Map Table (PMT): The PMT maps PID to a broadcast channel.
- Service Description Table (SDT): The SDT provides information to allow the receiver to associate applications with a data service. An application can specify logical channels and DTV programs that are of interest. The actual data sources can then be found via NRT and PMT.

The example in Figure 5 presents an application that views two DTV programs with the PIDs 30 and 32 and at the same time downloads a 3-D object from http://www.pidtv.com/. The application finds the channels where the programs can be accessed via the SDT at the top of the figure. If a channel is an Internet one, the NTR routes the request to the Internet. If a channel is a broadcast one, the PMT locates the PIDs in their physical channels.

Once programs are selected, the receiver receives the selected programs as they run. The selected programs may arrive simultaneously. A user may want to record and watch several sports programs at the same time. The receiver receives multiple audio/video (AV) data objects simultaneously, caches them on the local disk, and allows the viewer to watch any of the cached programs with a phase delay.

5.2 Resource Management

A receiver is equipped with a large and inexpensive disk for caching. The economical caching together with the random access capability of the disk enables a multimedia receiver to buffer a large amount of data to support time-shift operations. For AV data, a viewer can pause a live program to take a break from viewing while the broadcast stream continues arriving and being written to the local disk. The viewer can resume watching the program after the pause with a delay. He/She can also replay or fast-forward the program to get back in sync with

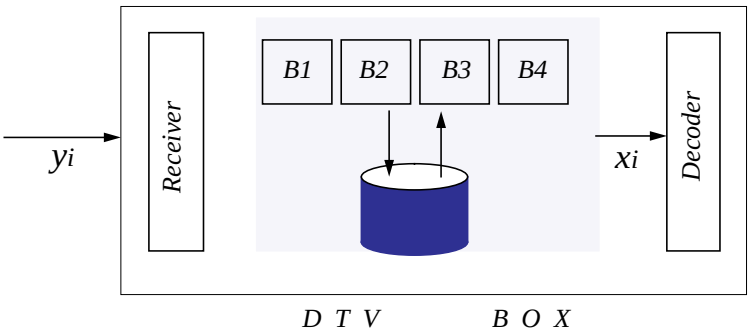


Figure 6: The Four-page Ping-pong Scheme.

the broadcast stream. For non-AV broadcast data, the receiver can stage the data on the disk, parse the data and then retain the data that are of interest (e.g., retain the score of a baseball game, the local weather, or the quotes of selected stocks).

Extending the DRAM cache with a large disk adds challenges to the receiver's buffer management. To ensure that IOs are efficiently scheduled, the receiver needs a *buffer manager* that can schedule the receiver's memory and disk bandwidth effectively to service a wide variety of interactive scenarios.

5.2.1 Buffer Management

We propose a new buffer-management scheme called "Four-page Ping-pong Scheme." This scheme is designed for AV data that have continuous constraints. However, since non-AV data may have to synchronize their playback time with AV data, they may be treated as continuous data with more liberal deadlines. For non-AV data that do not have playback deadline (e.g., software), we need just two pages for receiving data. In the remainder of this section, we assume that data are AV or related to a AV stream.

For the regular playback of a continuous data stream, the resource manager allocates four memory pages of equal size. These four pages are *ping-ponged* between the receiver and the processor (a decoder or an application may run on the processor) and are used to receive incoming data from the network, write data to the disk, read data from the disk, and provide data for decoding. As we will show shortly, the receiver and the processor each "owns" at most two pages at any time. The following example illustrates how the resource manager manages these four pages for a video playback.

At the start of the video playback, the resource manager allocates four pages, denoted by B_1, B_2, B_3 and B_4 (shown in Figure 6), for a stream. Once data begins arriving, the resource manager receives the data in one of the pages, say B_1. When B_1 is full, B_1 is available for decoding and hence the resource manager hands B_1 to the processor.[1] At the same time, the

[1] In this example, we assume that the playback starts once B_1 is filled up and is assigned to the processor. The playback can start earlier or later, depending on the difference between the data delivery and playback rates. For example, if the data delivery rate is substantially lower than the playback rate, the resource manager must

resource manager assigns another memory page, say B_2, to the receiver to continue receiving data. When page B_2 is full, the resource manager hands B_2 to the processor and assigns B_3 to the receiver to continue receiving data. At this time, the processor "owns" pages B_1 and B_2, and the receiver B_3.

One of two events occurs next: either the data in page B_1 is used up by the processor or page B_3 is filled up by the incoming packets. The resource manager takes different actions according to which event occurs first.

- If B_1 is used up first, the resource manager simply returns the page to the free pool.
- If B_3 is filled up first, the resource manager writes the page to the disk. Note that the processor does not need B_3 immediately. The processor needs only B_2 to safeguard a hiccup-free playback. When B_3 is being written to the disk, B_1 and B_2 are held by the processor. The resource manager must use page B_4 to receive incoming data. B_4 must be large enough for the disk to complete writing B_3 to disk, or else the receiver runs out of memory space to receive data once B_4 is full.

At this time, one of two things can happen next: the data in page B_1 is used up by the processor or page B_4 is filled up by the arriving data.

- When B_1 has been consumed by the processor, the resource manager starts consuming B_2. At the same time, the resource manager has to make sure that the data after that in B_2 is made available to the processor before the processor uses up B_2, or hiccups occur. If the processor uses up B_1 before B_3 is full, the resource manager assigns B_3 to the processor as soon as the page is filled up. If B_3 is filled up before page B_1 is consumed, an IO to write B_3 to the disk may be in progress or may have been completed. In the former case, the resource manager still makes B_3 available to the processor by ignoring the write (canceling the write may not be possible). If the write has been completed, the resource manager reads the data back from the disk into B_1 (the processor uses the data in B_1 after B_2 is consumed). Note that the size of page B_2 must be large enough to allow enough time for page B_1 to be replenished.
- If page B_4 is filled up first, the resource manager writes the data to the local disk. Since either page B_1 or B_3 must have been freed at this time, the resource manager uses the available page as the receiving buffer.

In summary, the resource manager is driven by two events: *page-consumed*, when the processor has consumed a page, and *page-full*, when the receiver has filled up a page. When the *page-consumed* event occurs, the resource manager may need to read a page of data from the disk before the processor uses up the next page. The page thus must be large enough to provide data to the processor before the read is completed. When the *page-full* event occurs, the resource manager may need to write the page to the disk to free up space for use. The page

accumulate enough data on disk before the playback can start to prevent display hiccups.

must be large enough so that during the write the receiver cannot fill up another page. This four-page ping-pong scheme has three benefits:

1. It serves as a rubber band to cushion the difference between the incoming and decoding bitrates.

2. It does not perform unnecessary memory-to-memory or memory-to-disk copying if the data is played back in real-time.

3. If the delay is desirable (a viewer pauses the playback), it uses the disk instead of memory to buffer data and limits the memory requirement to four pages. This ping-pong scheme minimizes memory requirement and hence caching cost for supporting regular playback and pause.

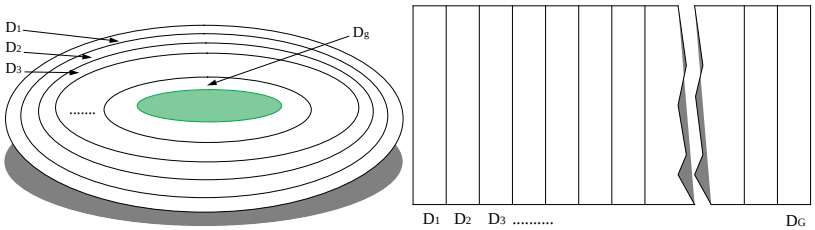
Supporting Fast-Scans

We call fast-forward and replay operations *fast-scans*. For both AV and conventional textual data, fast-scans are possible. For AV data, a fast-scan plays back a media stream at K times its encoding rate (e.g., $K = 4, 8, and\ 16$) in either a forward direction (fast-forward) or a backward direction (replay). To support a K times fast-scan, the receiver may have to supply data to the processor at K times the regular rate.[2] The processor then decodes and displays one out of every K frames. For processing conventional data, the receiver may want to speed up IOs K times even the data do not synchronize with AV data (e.g., during skimming a powerpoint presentation). Since the data are cached on the local disk, the receiver must perform disk reads at a K-time rate and needs memory to stage these data. The analysis of the memory requirement for supporting fast-scan is documented in Appendix A.

5.2.2 Data Placement

Minimizing disk-arm movement provides two benefits. First, it reduces disk latency and may reduce the memory requirement for staging continuous data. Second, being able to retrieve data fast from disk reduces the hiccup time. One can reduce disk latency by employing a disk partitioning scheme [11]. To shorten the distance that the disk-arm must travel, one can restrict the disk-arm movement for each IO to a smaller distance. Figure 7(a) shows that a disk is partitioned into G concentric cylinder groups, each of which occupies an average of $\frac{CYL}{G}$ cylinders. Figure 7(b) shows the cylinder groups presented in a matrix form, where each column represents a cylinder group. If we place the media data of a DTV program in one cylinder group, we can bound the worst-case seek distance by CYL/G cylinders. For instance,

[2] If the receiver does not parse the stream to learn where the offset of each frame is located, the receiver must read in the data sequentially, parse meta-data, then skip to the desired frame. Alternatively, the receiver can pre-parse the stream before storing the data on the disk, and build an index table for fast-scans. This pre-parsing may allow the receiver to avoid reading all blocks during a fast-scan operation. However, processing conventional data can also require fast IOs, where data blocks cannot be skipped.



(a) Disk Partitions (b) Disk Matrix

Figure 7: Disk Partitions

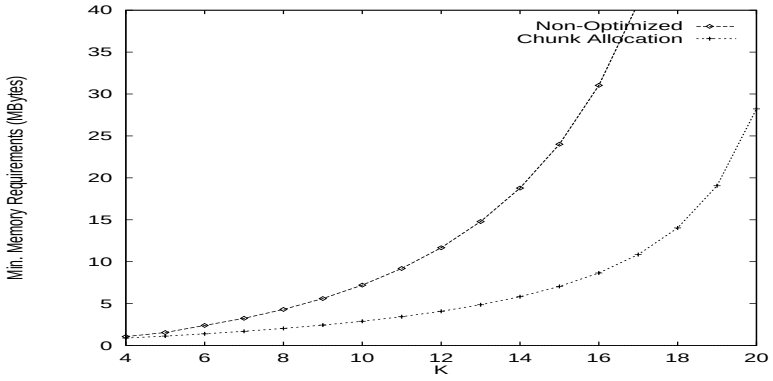


Figure 8: Memory Requirement vs. K

we can divide a 25-GByte hard disk into 15 cylinder groups, each of which can hold a DTV program of up to 30 minutes. The worst-case seek distance is reduced to 7% of the disk radius.

Figure 8 presents our experimental result on memory requirement using a Quantum Fireball disk. The x-axis of the figure indicates the speedup of the fast-scan and the y-axis shows the minimum memory required to support the K-time fast-scan. The upper curve in the figure shows the memory requirement computed by following the computation model in Appendix A. This curve shows that the memory requirement of the non-optimized scheme that we have presented grows without bound when K approaches 16. Employing the disk partitioning scheme mentioned above significantly reduces the memory requirement as indicated in the lower curve in the figure. For instance, when $K = 16$, the DRAM requirement is less than 10 MBytes. This low memory cost makes the receiver affordable, or put it the other way, given a fixed amount of memory, the receiver can stage more data in its main memory for performance gain.

5.3 Maximizing QoS

Given some data objects with their characteristics, the receiver's resources may not be adequate to support a particular interactive scenario or a data processing request that requires multiple data objects. Therefore, it is critical for the resource manager to be adaptive to the resource constraints and to degrade, if degradation is unavoidable, in a graceful manner. Given limited

memory and disk bandwidth, the design objective of a receiver is to prioritize resource allocation in order to maximize the user's satisfaction.

To measure a user's satisfaction, we must allow the user to have a say about what is important to the user and what is not. We thus associate each data object with a *QoS parameter*. A viewer can assign a QoS value to each data object, either implicitly or explicitly. For instance, if a user decides to turn off all interactive features, the resource manager can assign a QoS factor of zero to data objects that are needed to support interactive features. Regarding the data objects that are needed to support the playback, a higher QoS can be assigned to mission-critical data objects, such as broadcast video and audio streams and a lower QoS can be assigned to optional data objects such as captions and applets.

Given N_{all} requested data objects, the goal of the resource manager is to schedule N data objects ($N \leq N_{all}$) to maximize the total QoS under the resource constraint. We use α_i to denote the QoS requirement assigned to the ith data object ($0 \leq \alpha_i \leq 1$). Let j represent the scheduling order for N data objects. We would like to schedule the service to these N data objects in an order that maximizes the sum of the QoS. The objective function can be written as

$$Max \sum_{j=1}^{N} \alpha_j \times f_j(t - \sum_{k=1}^{j} \tau_k), \tag{10}$$

which is subject to the memory constraint

$$\sum_{j=1}^{N} S_j \leq M_{avail}.$$

We present an example in Appendix B, which illustrates how the resource manager schedules work under resource constraints.

In this section we show how to maximize QoS by selecting the desired objects to process. The scheme can be more effective if a data broadcaster can provide multiple versions of the same data object with different QoS. For instance, a video object can be temporally or spatially degraded to reduce its size. A smart receiver can then select a version that best fits its resource constraints to receive. The study of [19] presents a content adaptation (i.e., transcoding and summarization) framework that can be employed by a content provider to assist client adaptation.

6 Conclusion

In this paper, we discussed the potential and importance of data broadcasting as a data delivery mechanism. We showed why the database community may be interested in extending its research into this area. We presented challenges that data broadcasting faces and discussed some of our preliminary ideas. In particular, we focused our discussion on three areas: how to characterize different data objects, how a server schedule its broadcast and how a receiver manages its resources intelligently. Data broadcasting is still in its infancy. We believe that much work will be necessary to make this data transport an effective one.

Acknowledgements

We would like to thank professors Hector Garcia-Molina and Patrick M. Hanrahan at Stanford University for their helpful comments and suggestions.

References

[1] http://gartner4.gartnerweb.com/dq/static/about/press/pr-b200008.html. Feb. 2000.

[2] S. Acharya, M. Franklin, and S. Zdonik. Balancing push and pull for data broadcast. *ACM Sigmod*, May 1997.

[3] E. Chang. πDTV: A receiver-based interactive DTV architecture. *Proceedings of the 7th ACM Multimedia Conference*, November 1999.

[4] E. Chang and H. Garcia-Molina. Bubbleup - Low latency fast-scan for media servers. *Proceedings of the 5th ACM Multimedia Conference*, pages 87–98, November 1997.

[5] E. Chang and H. Garcia-Molina. Effective memory use in a media server. *Proceedings of the 23rd VLDB Conference*, pages 496–505, August 1997.

[6] M.-S. Chen and P. Yu. Storage and retrieval methods to support fully interactive playout in a disk-array-based video server. *ACM Multimedia Systsmes*, 3(3):126–35, July 1995.

[7] I. Dalgic and F. Tobagi. Characterization of quality and traffic for various video encoding schemes and various encoder control schemes. *Stanford Technical Report CSL-TR-96-701*, August 1996.

[8] I. Dalgic and F. Tobagi. Performance evaluation of ethernets and atm networks carrying video traffic. *Stanford Technical Report CSL-TR-96-702*, August 1996.

[9] M. Franklin and S. Zdonik. Data in your face: Push technology in perspective. *ACM Sigmod*, June 1998.

[10] S. Ghandeharizadeh. Continuous retrieval of multimedia data using parallelism. *IEEE Transactions on Knowledge and Data Engineering*, 5(4):658–69, 1993.

[11] S. Ghandeharizadeh, S. Kim, and C. Shahabi. On configuring a single disk continuous media server. *Sigmetrics Performance Evaluation*, 23(1):37–46, May 1995.

[12] K. Hua and S. Sheu. Skyscraper broadcasting: A new broadcasting scheme for metropolitan video-on-demand systems. *ACM Sigcomm*, pages 89–101, Sept. 1997.

[13] T. Johnson and A. Zhang. A framework for supporting quality-based presentation of continuous multimedia streams. *Proceedings of the 4th IEEE Conference on Multimedia Computing and Systems*, pages 169–176, June 1996.

[14] M. Klein, T. Ralya, B. Pollak, and R. Obenza. *A Practitioner's Handbook for Real-Time Analysis: Guide to Rate-Monotonic Analysis for Real-Time Systems.* Kluwer Academic Publishers, 1993.

[15] R. Ng and J. Yang. Maximizing buffer and disk utilizations for news on-demand. *Proceedings of the 20th VLDB Conference*, pages 451–462, 1994.

[16] B. Ozden, A. Biliris, R. Rastogi, and A. Silberschatz. A low-cost storage server for movie on demand databases. *Proc. VLDB*, September 1994.

[17] B. Ozden, R. Rastogi, and A. Silberschatz. A framework for the storage and retrieval of continuous media data. *Proc. IEEE Multimedia*, pages 2–13, May 1995.

[18] L. Sha, R. Rajkumar, and S. Sathaye. Generalized rate monotonic scheduling theory, a framework of developing real-time systems. *Proceedings of The IEEE*, January 1994.

[19] J. Smith, R. Mohan, and C.-S. Li. Scalable multimedia delivery for pervasive computing. *Proceedings of ACM Multimedia*, pages 131–140, November 1999.

[20] Y. Wang, J. Liu, D. Du, and J. Hsieh. Efficient video file allocation schemes for vod services. *ACM Multimedia Systems*, 5(4), September 1997.

Appendix A — Fast-Scan Memory Requirement

To analyze the receiver's use of main memory, our design goal is to avoid, as much as possible, display hiccups due to variability and delays in the input stream. Studies have shown that even just losing 0.1% of data can cause significant degradation in display quality, resulting from inter-frame decoding dependencies [7, 8]. Therefore, we take a conservative approach, that is, we assume the worst-case disk latency as well as peak data consumption and input rates.

Let K be the speedup of the playback (one can consider regular playback as a $K = 1$ fast-scan). Let T_r denote the longest time it takes to complete a K-page read. The maximum amount of data that the decoder can consume in T_r, denoted by $Max_x(T_r)$, can be expressed as

$$Max_x(T_r) = \max_{0 \le \tau \le tt} \sum_{i=\tau}^{\tau + \lceil \frac{K \times T_r}{\delta} \rceil - 1} x_i,$$

where tt denotes the total number of frame periods in x. Note that the total input size must be at least $Max_x(T_r)$ or the decoder may run out of data during the T_r period, causing a hiccup. We can express the minimum size of each of the K pages as

$$\sigma_r \ge \frac{Max_x(T_r)}{K}. \tag{11}$$

To conserve memory, we take the equality in the above expression, yielding

$$\sigma_r = \frac{Max_x(T_r)}{K}. \tag{12}$$

Similarly, let T_w denote the longest time the disk takes to complete writing out a page of data. The maximum amount of data that can possibly arrive in T_w time at the receiver can be expressed as

$$Max_y(T_w) = \max_{0 \le \tau \le tt} \sum_{j=\tau}^{\tau + \lceil \frac{T_w}{\delta} \rceil - 1} y_j.$$

The minimum page size, σ_w, is given by

$$\sigma_w = Max_y(T_w). \tag{13}$$

The page size selected must be the larger of the two, or $B = Max\ \{\sigma_r, \sigma_w\}$.

The available memory M_{avail} must be able to stage the incoming data (one page) and the data being consumed by the decoder (K pages). At the same time, the receiver may have one-page pending write and K-page pending read. The minimum available memory for supporting a K-time fast-scan thus must satisfy

$$M_{avail} \ge 2 \times (1 + K) \times B = 2 \times (1 + K) \times Max\ \{\sigma_r, \sigma_w\}. \tag{14}$$

If we do not have available the complete x_i and y_i functions, we may use bounds that give us the maximum data that can arrive (or can be consumed) per frame period. (These bounds can

O_i	S_i	$f_i(t)$	α_i	τ_i
O_1	3 MBytes	$f_b(t)$	1	20 ms
O_2	5 KBytes	$f_a(t)$	1	15 ms
O_3	1.2 MBytes	$f_b(t)$	0.3	18 ms

Table 5: Example Data Objects and their Characteristics

replace x_i and y_i in the computation of $Max_{y_i}(T)$ and $Max_{x_i}(T)$, respectively.) Such bounds may be obtainable from the intrinsic limits of the network and the decoding scheme (e.g., the DTV and the HDTV bitrate are capped at 6.4 and 19.2 Mbps, respectively). They can be rather conservative, since as we will see later, relatively small amounts of main memory can handle large network and consumption rates.

Next, we derive the formulas to compute T_r and T_w so that we can solve for M_{avail}. Since reads and writes can interleave, a read request may not be scheduled until a write completes and vice versa. Both T_w and T_r therefore must account for the worst-case delay before a read and write request can be completed. Fortunately, since the receiver does not issue a read when a read is pending and it does not issue a write when a write is pending, a read request waits at most for a write request to complete, and vice versa.

Assume that data pages may not be physically contiguous on the disk. (We discuss data placement policies in Section 5.2.2 to improve IO performance.) A round of IO time for a read and write pair is

$$T = (K + 1) \times (\gamma(d) + B/TR), \tag{15}$$

which includes reading one and writing K pages. Each page IO takes $\gamma(d)$ seek time and B/TR data transfer time. Finally, setting T_r and T_w equal to T guarantees two things: the arriving data will not overflow the buffer and the data supply to the decoder will not underflow the buffer.

Given x_i, y_i, M_{avail} and the disk parameters, one can solve for the maximum K the receiver can support using Equations 2, 3, 4 and 5. One can also solve for the memory requirement given x_i, y_i, K and the disk parameters.

Appendix B — A QoS-Based Scheduling Example

Suppose a DTV box needs to process three data objects: O_1, O_2, and O_3. Suppose these objects have the characteristics depicted in Table 5 and the DTV box has $M_{avail} = 4$ MBytes of DRAM. To simplify our discussion, we precompute the latency τ_i (the IO time to transfer S_i amount of data) for each object in the table. Suppose each frame period is 33 ms. The latency functions $f_a(t)$ and $f_b(t)$ are depicted in Figure 9.

With the 4 MBytes DRAM constraint, the resource manager cannot process all three data objects since doing so requires 4.205 MBytes of DRAM. The resource manager can, however, process any two objects in any orders, and it thus has six feasible schedules: $\{O_1, O_2\}$, $\{O_2, O_1\}$, $\{O_1, O_3\}$, $\{O_3, O_1\}$, $\{O_2, O_3\}$, and $\{O_3, O_2\}$.

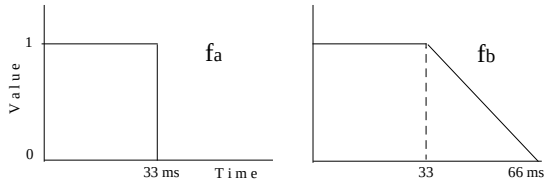


Figure 9: Latency Functions $f_a(t)$ and $f_b(t)$

Schedule	τ_1	$\tau_1 + \tau_2$	*Total QoS*
$\{O_1, O_2\}$	20 ms	35 ms	1
$\{O_2, O_1\}$	15 ms	35 ms	1.94
$\{O_1, O_3\}$	20 ms	38 ms	1.28
$\{O_3, O_1\}$	18 ms	38 ms	1.15
$\{O_2, O_3\}$	15 ms	33 ms	1.3
$\{O_3, O_2\}$	18 ms	33 ms	1.3

Table 6: QoSs of Feasible Schedules

To compute the total quality that a schedule can achieve, we plug all given statistics in Table 5 into Equation 2. For instance, for the first schedule $\{O_1, O_2\}$, the resource manager performs an IO to read data for O_1 then O_2. The latency to perform IO for O_1 is 20 ms and the latency function $f_b(t-20)$ returns one. However, when the IO is completed for O_2, the latency function $f_a(t-35)$ returns zero. (The excessive delay of retrieving O_2 makes the object worthless to the viewer.) The total QoS is thus $1 \times 1 + 1 \times 0 = 1$. If we switch the service order to first O_2 and then O_1, the latency for reading O_2 is 15 ms and the latency function $f_a(t-15)$ returns one. When the IO is completed for O_1, the latency function $f_b(t-35)$ returns 0.94. The total QoS is 1.94, a better total QoS than we get by processing first O_1 and then O_2. Table 6 lists the total QoSs for all six feasible schedules. The schedule that achieves the maximum QoS is $\{O_2, O_1\}$. In other words, under the resource constraint, retrieving first O_2 and then O_1 achieves the maximum QoS.

Note that different clients may receive the same data objects, but based on the QoS factors the clients assign to these objects and on the local resource constraints, the resource manager allocates and schedules resources adaptively to achieve different quality results. In the example above, if we assign to the audio object a QoS factor of 0.3 and to the image a QoS factor of 1.0, the optimal schedule becomes $\{O_1, O_3\}$. Similarly, changing the latency functions and the resource constraints also alters the optimal schedule. Therefore, the resource management scheme we propose is adaptive to user preferences, data characteristics, and resource constraints.

Solving the Optimal Schedule

A brute-force method to solve the optimal schedule is to enumerate all possible permutations of N_{all} requests and select the permutation that produces the maximum total QoS. This approach, however, can be very expensive if N_{all} is large. For example, when $N_{all} = 5$, a probable scenario,

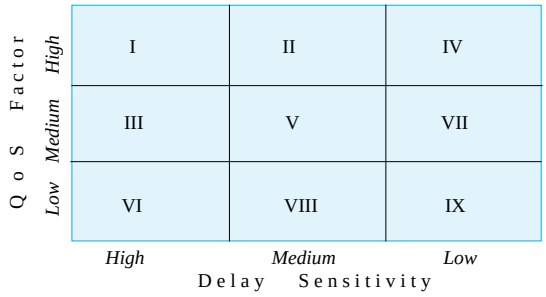


Figure 10: IO Scheduling Table

the total number of permutations is 120 (roughly on the order of $O(N_{all}^2)$). Since we do not want the computation of the optimal schedule itself to become a delay factor, we use an efficient method that has a linear cost (i.e., the cost of computation is on the order of $O(N_{all})$) to approximate the result.

To reduce the computation cost, we put the α values into three categories: low, medium, and high. We also put the delay sensitivity of an object into these three categories. We then plot N_{all} data objects in the schedule table as shown in Figure 10. The x-axis of the figure is the delay sensitivity and the y-axis the QoS factor. After N_{all} IO requests find their quadrants in the schedule table, the resource manager schedules the IOs from quadrant I to IX in ascending order. This method first effectively separates the highest and the lowest priority tasks from the rest and then the resource manager schedules the highest priority tasks first and abandons the lowest ones if necessary. Why does this work? Suppose we define high as 0.8, medium as 0.5 and low as 0.2. It is very clear that a task in the high-high quadrant enjoys a substantially higher contribution to the total QoS than a task in the low-low quadrant ($0.8 \times 0.8 = 0.64$ versus $0.2 \times 0.2 = 0.04$). For the remaining tasks, the scheduler gives the tasks with the higher α values (the tasks of greater importance to the viewer) higher scheduling priorities. If the remaining tasks cannot be separated, one can divide the values on both axes into finer grains. Since a viewer can focus on only a limited number of data objects, we believe that this scheme can work well.